“Linux Process Tree Visualizer — Live Process Hierarchy Using the Linux /proc Filesystem”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINNERING

By

HANSIKA REDDY GUNAPATI	**2520030237**
TEJA HASINI REDDY	**2520030048**
THANAM ANTHONY NISSY	**2520030064**
SIMRAN JANGID	**2520030166**

Under the Esteemed Guidance of

Dr. Vadlamudi Muniraju Naidu

Associate Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

Declaration

The Project Report entitled **"Linux Process Tree Visualizer — Live Process Hierarchy Using the Linux /proc Filesystem"** is a record of bonafide work prepared by the students listed below in partial fulfillment of the requirements for the award of B. Tech in Computer Science and Engineering to K L University. The report describes the supplied C implementation and does not claim runtime test results that have not been recorded. The results embodied in this report have not been copied from any other department, university, or institute.

HANSIKA REDDY GUNAPATI	**2520030237**
TEJA HASINI REDDY	**2520030048**
THANAM ANTHONY NISSY	**2520030064**
SIMRAN JANGID	**2520030166**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the project report entitled **"Linux Process Tree Visualizer — Live Process Hierarchy Using the Linux /proc Filesystem"** has been prepared by the students listed below in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in the Department of Computer Science Engineering, K L (Deemed to be) University, during the academic year **2026-2027**. Signature and approval fields are left blank for the appropriate authorities.

HANSIKA REDDY GUNAPATI	**2520030237**
TEJA HASINI REDDY	**2520030048**
THANAM ANTHONY NISSY	**2520030064**
SIMRAN JANGID	**2520030166**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Dr. Vadlamudi Muniraju Naidu, Associate Professor, Department of Computer Science and Engineering, for guidance and encouragement during the preparation of this Operating Systems and Systems Programming project.

We thank the faculty and staff of the Department of Computer Science Engineering for providing an academic environment in which we could study Linux process management and systems programming. We also acknowledge the support of our parents, friends, and classmates.

This report uses the supplied C source code, presentation, and sample documentation. Execution screenshots and measured results are to be inserted only after genuine testing in a Linux environment.

HANSIKA REDDY GUNAPATI **2520030237**
TEJA HASINI REDDY **2520030048**
THANAM ANTHONY NISSY **2520030064**
SIMRAN JANGID **2520030166**

Abstract

Linux Process Tree Visualizer is a command-line systems-programming project written in C that reads process metadata from the Linux /proc filesystem and displays parent-child relationships as a terminal tree. It collects PID, PPID, process name, and state from /proc/<pid>/stat, sorts records by PID, identifies roots, and recursively renders children whose PPID matches a parent PID. The implementation uses directory and file operations, strtol(), sscanf(), qsort(), ANSI escape sequences, sigaction(), and nanosleep(). It supports --all, --once, and --help. By default it filters records whose PID is 2 or whose PPID is 2. The project demonstrates process identifiers, process states, procfs metadata, defensive parsing, recursion, and signal-based shutdown. Limitations include a fixed capacity of 20,000 records, non-atomic snapshots, permission-related failures, repeated linear scans, recursion depth limits, terminal display constraints, and a fixed refresh interval. No runtime results are claimed because execution logs and genuine screenshots were not supplied.

Index

Section	Reference
Declaration	i
Certificate	ii
Acknowledgement	iii
Abstract	iv
List of Figures	v
List of Tables	vi
List of Abbreviations	vii
Chapter 1	5
Chapter 2	9
Chapter 3	13
Chapter 4	17
Chapter 5	21
Chapter 6	25
Chapter 7	29
Chapter 8	33
Chapter 9	37
Chapter 10	41

Page references are indicative navigational aids and should be refreshed after substantial editing or reflow.

List of Figures

Figure	Caption
Figure 1	System architecture
Figure 2	Program flowchart
Figure 3	Illustrative process tree — synthetic data
Figure 4	Implemented process-state color mapping

List of Tables

Table	Caption
Table 1	Objectives mapped to implementation
Table 2	System requirements
Table 3	Header files
Table 4	Process record fields
Table 5	Module summary
Table 6	Complexity analysis
Table 7	Test plan
Table 8	Limitations and improvements

List of Abbreviations

Abbreviation	Meaning
ANSI	American National Standards Institute
GCC	GNU Compiler Collection
PID	Process Identifier
PPID	Parent Process Identifier
POSIX	Portable Operating System Interface
CLI	Command-Line Interface
CPU	Central Processing Unit
SIGINT	Interrupt signal
SIGTERM	Termination request signal
/proc	Linux process and kernel information pseudo-filesystem

CHAPTER - 1 INTRODUCTION

1.1 Background of the Project

Linux runs interactive shells, desktop applications, background services, and many other processes concurrently. Each process has a process identifier (PID) and a recorded parent process identifier (PPID). The relationships between these identifiers form a hierarchy that helps explain how commands and services were launched. A flat list of processes can show individual records, but does not immediately communicate the ancestry connecting them. The Linux Process Tree Visualizer reads process metadata from the virtual /proc filesystem and presents parent-child relationships as a readable terminal tree. The program is implemented in C and is intended to connect operating-system theory with practical systems programming.

1.2 Problem Statement

The /proc directory contains numeric entries for processes, but its raw contents are not a ready-made hierarchy. The set of entries can change while it is being scanned: a process may terminate before its stat file is opened, or a read may fail. The problem is to collect a bounded snapshot defensively, parse PID, PPID, name, and state, reconstruct relationships, and display the hierarchy in a form that remains understandable and can be interrupted cleanly.

1.3 Motivation

The project makes process relationships observable on a real Linux system. It demonstrates how kernel-provided metadata can be inspected from user space, why a process name must be parsed carefully, and how recursion can turn a flat record array into a tree-shaped display. It also gives students a compact example of file operations, sorting, string handling, signal handling, and timed waiting.

1.4 Scope of the Project

The application reads /proc/<pid>/stat, stores process metadata, sorts records by PID, finds roots, recursively prints children, highlights R/T/Z states, and refreshes every three seconds in live mode. It supports --all, --once, and --help. The default filter skips records whose PID is 2 or whose PPID is 2. The program does not control processes, implement a GUI, store a database, use networking, or launch worker threads.

1.5 Objectives of the Project

The objectives are to discover numeric process directories; parse PID, PPID, process name, and state; reconstruct parent-child relationships; provide a readable tree; show selected process-state counts; support both snapshot and live modes; and stop the refresh loop when SIGINT or SIGTERM is received. The project also aims to make the source understandable enough for viva discussion.

1.6 Importance of the Application

The hierarchy can help a learner understand shell ancestry, services, and child tasks. The project is primarily educational and does not replace mature tools such as ps, pstree, or top.

Its value lies in exposing the implementation steps—collection, parsing, sorting, matching, recursion, and output—in a relatively small source file.

1.7 Target Users and Audience

The intended audience includes undergraduate students studying Operating Systems and Systems Programming, instructors demonstrating process concepts, and beginner systems programmers learning Linux interfaces. Basic knowledge of C, a terminal, and process identifiers is useful.

1.8 Organization of the Report

Chapter 2 explains OS concepts; Chapter 3 lists requirements; Chapter 4 describes technologies; Chapter 5 explains architecture; Chapter 6 analyzes the code and algorithms; Chapter 7 documents compilation and test plans; Chapter 8 discusses limitations and future scope; Chapter 9 concludes; Chapter 10 provides references and appendices.

Objective	**Implementation element**
Discover processes	scan_processes() enumerates numeric /proc entries
Parse metadata	read_process() parses stat fields
Build hierarchy	print_node() matches PPID to PID
Readable output	branch glyphs, state colors, summary
Modes	--all, --once, --help
Graceful stop	signal handlers update running flag

Table 1. Project objectives mapped to implementation.

CHAPTER - 2 OPERATING SYSTEM CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Fundamentals

An operating system manages processor time, memory, filesystems, devices, and process execution. The kernel provides mechanisms through system calls and exposes selected information to user programs. This application runs in user space and reads kernel-provided process metadata rather than modifying kernel state.

2.2 Processes and Process Identifiers

A process is an executing program with an execution context and resources. Linux assigns a PID to identify it during its lifetime. PIDs can be reused after termination, so a PID should not be treated as a permanent identity across time. A process snapshot is only a view of records collected during a short interval.

2.3 Parent Process ID and Process Hierarchy

PPID records the parent process identifier. A child record belongs beneath a collected parent when child.ppid equals parent.pid. The source sorts records by PID for predictable ordering, but the hierarchy is created by matching PPID to PID. Sorting alone does not establish parent-child links.

2.4 Process States in Linux

The state character read from stat represents a process state at the time of reading. The program colors R (running) green, T (stopped) yellow, and Z (zombie) red. Other state characters are printed without these special colors. The header counts only R, T, and Z; it is not a complete summary of all Linux state categories.

2.5 Linux /proc Virtual Filesystem

/proc is a pseudo-filesystem that exposes information maintained by the kernel. Numeric directory names generally identify process entries. It is not a conventional disk directory of persistent application files. Since processes change concurrently, a directory entry can disappear between enumeration and file access.

2.6 /proc/<pid>/stat Metadata

The source reads one line from /proc/<pid>/stat into a 4096-byte buffer. It isolates the process name between the first opening parenthesis and the last closing parenthesis because names may contain spaces or parentheses. It parses state and PPID from the remainder of the line and validates the PID before storing it.

2.7 Process Creation and Termination

Processes may be created or terminate at any point during scanning. If fopen() or fgets() fails, the record is skipped. Parent and child information may therefore be incomplete in one snapshot. This is expected for a lightweight live reader that does not freeze the process table.

2.8 Process Trees and Recursive Traversal

The tree is printed recursively. Each node is emitted, then the program scans for unprinted records whose PPID matches the current PID. Branch glyphs show sibling structure. A printed flag prevents duplicates, and a depth guard returns when depth exceeds 1000.

2.9 Sorting and Searching

qsort() orders the records by PID. find_process() linearly searches for a parent PID, while count_children() and print_node() scan the array to locate children. These repeated scans can lead to $O(n^2)$ worst-case behavior, despite the sort itself typically taking O(n log n).

2.10 Signal Handling

SIGINT is commonly generated by Ctrl+C; SIGTERM requests termination. The handler only sets a volatile sig_atomic_t flag. Normal control flow observes the flag and exits the loop. The program avoids complex I/O inside the signal handler.

2.11 POSIX Interfaces

The program uses interfaces such as sigaction(), sigemptyset(), and nanosleep(), together with standard C file and directory functions. These interfaces can fail, so checking return values is important. The source checks key file-read operations but does not check every system-interface return value.

2.12 Existing Tools

ps provides configurable tabular process listings, top emphasizes periodic process/resource statistics, and pstree specializes in displaying process ancestry. These mature utilities are more feature-rich. This project is valuable as an educational implementation of the data collection and tree-rendering pipeline, not as a replacement.

2.13 Proposed Approach

The program parses command-line flags, installs signal handlers, scans numeric /proc entries, parses metadata, applies the PID 2 / PPID 2 filter unless --all is used, sorts by PID, identifies roots, recursively prints the hierarchy, displays R/T/Z counts, and either exits or waits three seconds before repeating.

2.14 Advantages

The application has a small runtime footprint, no database dependency, readable source code, and a clear terminal output. Defensive file checks allow it to continue through transient failures. Its limitations include fixed capacity, repeated scans, recursion depth, limited state highlighting, and terminal-dependent output.

CHAPTER - 3 SYSTEM REQUIREMENTS

3.1 Hardware Requirements

No specialized hardware is required. A conventional computer capable of running Linux and a terminal is sufficient. Exact CPU and memory requirements depend on the host's process population. No benchmark machine or measured resource result was supplied, so the report makes no numeric performance claim.

3.2 Software Requirements

Required: Linux with /proc available, a C compiler compatible with the source, and a terminal. GCC is the documented compiler. A text editor or IDE is optional. Git and diagramming software may help development but are not runtime dependencies.

3.3 Operating System Requirements

The application depends on Linux /proc process metadata and POSIX interfaces. It is not a native Windows program. It may run in a Linux virtual machine or subsystem if that environment exposes a suitable /proc filesystem and supports the required interfaces.

3.4 Compiler Requirements

The supported command is `gcc -Wall -Wextra -O2 -o ptree ptree.c`. The warning options enable diagnostics, -O2 requests optimization, and -o names the executable. The source defines _POSIX_C_SOURCE 200809L before its headers.

3.5 Development Environment

A working directory, Linux shell, GCC, and terminal are sufficient. Actual local execution evidence should be captured before submission; this report does not claim that compilation was executed in the user's environment.

3.6 Libraries and Headers

ctype.h provides isdigit(); dirent.h provides directory operations; signal.h provides signal interfaces; stdbool.h defines bool; stdio.h provides file and output functions; stdlib.h provides strtol() and qsort(); string.h provides string operations; time.h provides timespec and nanosleep(); unistd.h provides POSIX declarations.

3.7 Minimum and Recommended Configuration

Minimum: Linux with /proc, a compatible compiler, and a terminal. Recommended: a modern Linux environment, GCC, and a terminal supporting UTF-8 box-drawing glyphs and ANSI colors. The fixed process array stores at most 20,000 records in one scan.

Category	Requirement	Status
OS	Linux with /proc and POSIX interfaces	Required
Compiler	GCC or compatible C compiler	Required

Category	Requirement	Status
Terminal	UTF-8 glyphs and ANSI escapes	Recommended
Editor/IDE	Any C editor	Optional
Capacity	20,000 stored records maximum	Implemented limit

Table 2. System requirements and constraints.

CHAPTER - 4 TECHNOLOGIES AND TOOLS USED

4.1 C Programming Language

C is used for fixed-size records, pointer-based string processing, arrays, and direct access to POSIX interfaces. The source is organized into helper functions that separate reading, scanning, sorting, rendering, and control flow.

4.2 Linux Operating System

Linux provides process identifiers, process states, the /proc filesystem, and signal/timing behavior. The program is a user-space observer and does not alter the processes it lists.

4.3 /proc Filesystem

opendir() opens /proc, readdir() enumerates its entries, and closedir() releases the directory stream. The scanner accepts only names consisting entirely of decimal digits as candidate PIDs.

4.4 GCC

GCC compiles ptree.c into ptree. -Wall and -Wextra enable warnings, while -O2 enables optimization. A real compilation result must be recorded locally.

4.5 Standard C Library

printf(), snprintf(), strchr(), strrchr(), memcpy(), strtol(), sscanf(), and qsort() support output, path construction, parsing, copying, conversion, and ordering.

4.6 Directory Operations

opendir() returns a directory stream or NULL. readdir() returns entries until exhausted. closedir() closes the stream. If /proc cannot be opened, the program reports the error using perror() and returns from the scan.

4.7 File Operations

fopen() opens each stat file, fgets() reads a bounded line, and fclose() closes the file. Failed opens or reads cause the record to be skipped, accommodating processes that disappear during scanning.

4.8 Parsing Functions

strtol() converts the numeric directory name and the result is range-checked. sscanf() extracts state and PPID from the text after the final closing parenthesis and the return count is checked. The prior digit-only test narrows the accepted PID strings.

4.9 qsort()

qsort() orders the array in place by PID using compare_processes(). This improves deterministic output but does not create hierarchy links.

4.10 sigaction()

A sigaction structure is configured with handle_signal() for SIGINT and SIGTERM. The handler sets running to zero. The source does not check every return value from signal setup calls, which is a potential hardening improvement.

4.11 nanosleep()

The live loop sleeps for REFRESH_SECONDS, defined as three seconds. If interrupted, nanosleep() returns -1 and the code retries with the remaining delay while running is true. There is no supported --interval option.

4.12 ANSI Escape Sequences

ESC[2J clears the screen and ESC[H moves the cursor to the top-left. Green, yellow, and red sequences highlight selected states. Terminals differ in ANSI and Unicode support.

4.13 Command-Line Arguments

The supported options are --all, --once, and --help. Unknown options print an error and usage information, then return status 1. The --interval option shown in the presentation is not implemented.

Header	Role
ctype.h	isdigit() numeric-name check
dirent.h	opendir/readdir/closedir
signal.h	sigaction and sig_atomic_t
stdbool.h	bool type
stdio.h	FILE and formatted I/O
stdlib.h	strtol and qsort
string.h	string search/copy
time.h	timespec and nanosleep
unistd.h	POSIX declarations

Table 3. Header files and their roles.

CHAPTER - 5 SYSTEM ARCHITECTURE AND DESIGN

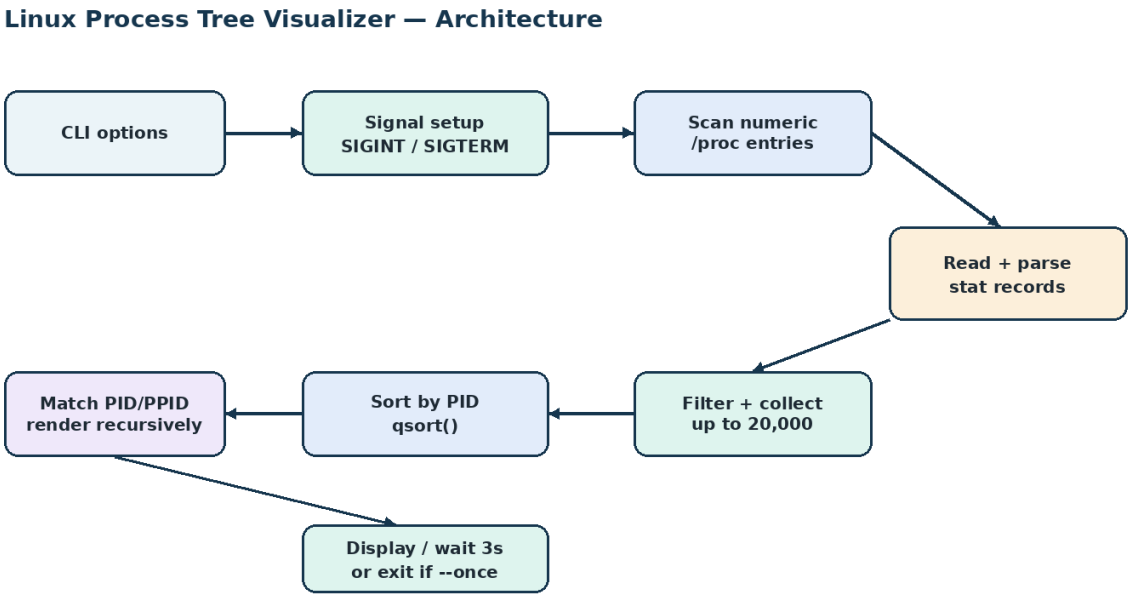


Figure 1. System architecture of the Linux Process Tree Visualizer.

5.1 Overall Architecture

The application is a single-process command-line pipeline. Options are parsed, signal handlers are installed, /proc is scanned, records are parsed and filtered, records are sorted, roots and children are matched, and the tree is printed. Live mode waits three seconds and repeats.

5.2 Architecture Diagram

Figure 1 shows the real pipeline from CLI options through signal setup, scanning, parsing, filtering, sorting, recursive rendering, and refresh. The --once path exits after one display; --help exits before signal setup.

5.3 System Workflow

Each cycle resets process_count, opens /proc, visits numeric names, parses stat files, applies filtering, stores valid records, closes the directory, and sorts the array. display_tree() counts states, prints a header, emits roots, and then prints any unprinted records as a fallback.

5.4 Data Flow Diagram

Input is /proc and its process stat files. Parsing creates Process records. Filtering may discard selected records. Sorting orders the records. The renderer matches PPID to PID and produces terminal text, state colors, and summary counts.

5.5 Program Flowchart

The flowchart includes option parsing, help exit, signal setup, scan/display loop, --once decision, three-second nanosleep(), and signal-driven termination. Unknown arguments print usage and exit with a non-zero status.

5.6 Module Decomposition

Logical modules are signal handling, helpers, stat reading, process scanning, tree printing, usage output, and main control flow. They are functions within one C source file, not separately compiled modules.

5.7 Process Record

Each record stores pid, ppid, name[256], state, and printed. There is no stored depth field; indentation is managed through recursive calls and a prefix string. The depth parameter acts as a recursion guard.

5.8 Discovery and Collection

The scanner checks that every character in the directory name is a digit, applies a capacity check, attempts to read the stat record, applies the default filter, and appends successful records.

5.9 Sorting and Matching

qsort() orders by PID. Root detection calls find_process() to see whether a parent record exists. Child discovery scans for PPID values equal to the current PID. These relationships—not the sort—create the hierarchy.

5.10 Recursive Rendering

print_node() checks depth and printed, marks the node, prints the record, counts unprinted children, constructs the next prefix, and recursively prints matching children. The depth guard is 1000 and the prefix buffer is 4096 bytes.

5.11 State Color Mapping

R maps to green, T to yellow, Z to red, and all other states to reset. display_tree() separately counts only R, T, and Z records.

5.12 Refresh and Termination

Live mode clears the screen, displays a tree, waits three seconds, and repeats. The signal handler sets running to zero; main control flow then exits the loop and prints a stop message.

5.13 CLI Design

The usage format is `./ptree [--all] [--once] [--help]`. Options can be combined. There is no interactive prompt and no configurable interval.

5.14 Input and Output

The input is command-line flags and the current /proc view. Output includes mode, filter status, process count, R/T/Z counts, a legend, and the hierarchy.

Module	**Functions**	**Responsibility**
Control	main(), print_usage()	Options and loop
Signals	handle_signal()	Shutdown flag
Reader	read_process()	Parse stat
Scanner	scan_processes()	Collect records
Search/sort	compare_processes(), find_process()	Order and lookup
Renderer	count_children(), print_node(), display_tree()	Tree and summary

Table 5. Module summary.

CHAPTER - 6 IMPLEMENTATION AND ALGORITHMS

6.1 Program Structure

The global array stores Process records. read_process() parses a record, scan_processes() collects them, compare_processes() supports sorting, find_process() searches by PID, and print_node()/display_tree() render the output. main() manages flags, signals, and refresh.

6.2 Header Files

ctype.h: digit checks; dirent.h: directory enumeration; signal.h: signal APIs and sig_atomic_t; stdbool.h: bool; stdio.h: file/output functions; stdlib.h: strtol/qsort; string.h: string parsing; time.h: nanosleep/timespec; unistd.h: POSIX declarations.

6.3 Constants

MAX_PROCESSES is 20000; NAME_SIZE is 256; REFRESH_SECONDS is 3. The path buffer is 64 bytes, stat buffer 4096 bytes, and indentation prefix 4096 bytes. Fixed limits simplify storage but constrain extreme cases.

6.4 Process Structure

Fields are pid (int), ppid (int), name (char[256]), state (char), and printed (bool). The printed flag tracks output status, not the kernel process state.

6.5 Global Flags

process_count tracks the number of accepted records. running is volatile sig_atomic_t for signal communication. once selects a one-shot snapshot; show_kernel controls the PID 2 / PPID 2 filter.

6.6 Signal Handler

handle_signal() ignores the numeric argument and sets running to zero. It does not print, allocate memory, or traverse /proc. The normal program flow performs the shutdown output.

6.7 Comparator

compare_processes() compares PIDs with relational operators and returns -1, 1, or 0. It avoids subtraction-based comparison that could overflow.

6.8 Linear Lookup

find_process(pid) scans the array and returns the matching index or -1. It is O(n) time and O(1) auxiliary space.

6.9 State Color

state_color() returns a static ANSI string for R, T, Z, or the reset sequence for all other states.

6.10 Printing a Record

print_process() prints the process name, PID, PPID, and state character, placing the appropriate color sequence around the state.

6.11 Reading stat

read_process() builds /proc/<pid>/stat using snprintf(), opens it, reads one line, closes it, locates parentheses, validates the PID, copies the name with a bounded length, and parses state and PPID.

6.12 Field Extraction

The first '(' and last ')' delimit the process name. The copy length is capped at NAME_SIZE-1 and NUL-terminated. strtol() converts the PID, and sscanf() must return two conversions for state and PPID.

6.13 Numeric Directory Scan

Only entries with all-digit names are considered. If the array is full, later candidates are skipped. Unreadable or malformed entries are skipped, the directory is closed, and qsort() orders the records.

6.14 Transient Failures

If a process exits before its stat file can be opened or read, read_process() returns false. This avoids aborting the entire scan, but the resulting snapshot may be incomplete.

6.15 Kernel-Thread Filter

Unless --all is supplied, records with PID 2 or PPID 2 are discarded. This is the exact implemented rule, not a complete kernel-thread classifier for every Linux configuration.

6.16 Sorting

qsort() typically has O(n log n) time complexity. The full application is not O(n log n), because root detection and child rendering repeatedly scan the array.

6.17 Child Counting

count_children() scans all records and counts unprinted records with a matching PPID. It is O(n) per call and may be called for many nodes.

6.18 Recursive Rendering

print_node() returns if depth exceeds 1000 or a record is already printed. It prints the current node, calculates children, extends the prefix, and recursively prints children in sorted array order.

6.19 Root Detection

A record is treated as a root when PPID is non-positive or its parent is absent from the collected array. A parent can be absent due to filtering, permissions, namespace visibility, or a race.

6.20 Statistics

display_tree() counts R/T/Z states, optionally clears the screen, prints the header and legend, emits roots, and performs a fallback pass for unprinted records. fflush(stdout) flushes the output.

6.21 Live Refresh

main() scans and displays while running is true. --once breaks after the first display. Otherwise nanosleep() waits three seconds and may be retried with the remaining delay if interrupted.

6.22 Argument Processing

--once sets once; --all sets show_kernel; --help prints usage and returns 0. Unknown options print an error and usage, then return 1. --interval is unsupported.

6.23 Graceful Termination

SIGINT and SIGTERM set running to zero. The main loop stops and, in live mode, prints the final stopped message. The handler does not attempt to terminate the other processes shown in the tree.

6.24 Error Handling

The code checks /proc opening, stat-file opening and reading, parenthesis positions, PID range, and sscanf() conversion count. Return values from signal setup and some formatting calls are not checked comprehensively.

6.25 Pseudocode

SCAN: reset count; open /proc; for each entry check numeric name and capacity; read/parse stat; apply filter; append; close; sort by PID. DISPLAY: count R/T/Z; print header; find roots; recursively print children; fallback-print leftovers. PRINT_NODE: guard depth/printed; mark; print; count children; build prefix; recurse over matching PPIDs.

6.26 Complexity Analysis

Let n be the number of collected records. Scanning has O(n) loop-level work plus filesystem costs. qsort() is typically O(n log n). find_process() and count_children() are O(n). Root detection may call linear lookup for many records, and recursive rendering repeatedly scans all records, so the worst-case display phase is $O(n^2)$. Storage is bounded by the fixed process array; recursion and prefix storage depend on depth.

6.27 Code Review Notes

The source uses bounded buffers and a capacity check. The process name is copied with an explicit terminator. It tolerates transient file-read failures and uses a signal-safe flag. Possible hardening includes checking all signal setup return values, checking snprintf() truncation, reporting capacity truncation, and using strtol() end-pointer validation.

Operation	Complexity	Reason

Operation	Complexity	Reason
Scan/parse	O(n) loop work	Each entry visited once
qsort	Typically O(n log n)	PID ordering
find_process	O(n)	Linear lookup
count_children	O(n)	Full-array scan
Root detection	$O(n^2)$ worst case	Repeated lookup
Tree rendering	$O(n^2)$ worst case	Repeated scans
Overall display	$O(n^2)$ worst case	Linear scans dominate

Table 6. Complexity analysis.

Supplementary Implementation Analysis

Why snapshots are not atomic

The kernel does not freeze the process table while the program reads separate stat files. A process may appear after enumeration, disappear before a file is opened, or change state between reads. The output is therefore a best-effort snapshot assembled over time, not a transactional view.

Security and permissions

The program reads metadata without requesting elevated privileges. Procfs mount options, namespaces, and access controls can hide or restrict entries. The output reflects the view available to the running program, not necessarily every process on the physical host.

Reliability and input validation

The source checks numeric names, file opening, line reading, parentheses, PID range, and sscanf() conversion count. Potential improvements include checking all signal setup results, detecting snprintf() truncation, and using strtol() end-pointer validation.

Design trade-offs

A fixed array simplifies storage but limits the scan to 20,000 records. Recursive rendering is easy to understand but can use stack space and repeated scans. ANSI colors improve readability but depend on terminal support. These trade-offs are reasonable for a compact teaching project.

Test evidence discipline

A planned test is not proof of a passing test. Actual results remain Not Executed until the program is run and evidence captured. Record the environment, command, exit status, observation, and screenshot or transcript.

Possible data-structure improvements

A PID-to-index hash map and child adjacency lists could reduce repeated scans. The current source does not implement these structures. Any revised implementation should be evaluated separately and supported by measurements.

Terminal rendering

Unicode branch glyphs and ANSI control sequences depend on terminal encoding and behavior. Large trees may exceed the visible terminal height, and deep indentation can wrap. A future no-color or paginated mode could improve accessibility.

Command-line behavior

The parser supports only --all, --once, and --help. Help exits successfully; unknown options print an error and usage and return 1. Refresh interval is a compile-time constant.

State-count interpretation

Only R, T, and Z receive special colors and summary counts. Other state characters are still printed but are not counted in separate summary categories.

CHAPTER - 7 COMPILATION, EXECUTION, TESTING, AND RESULTS

Program Flowchart

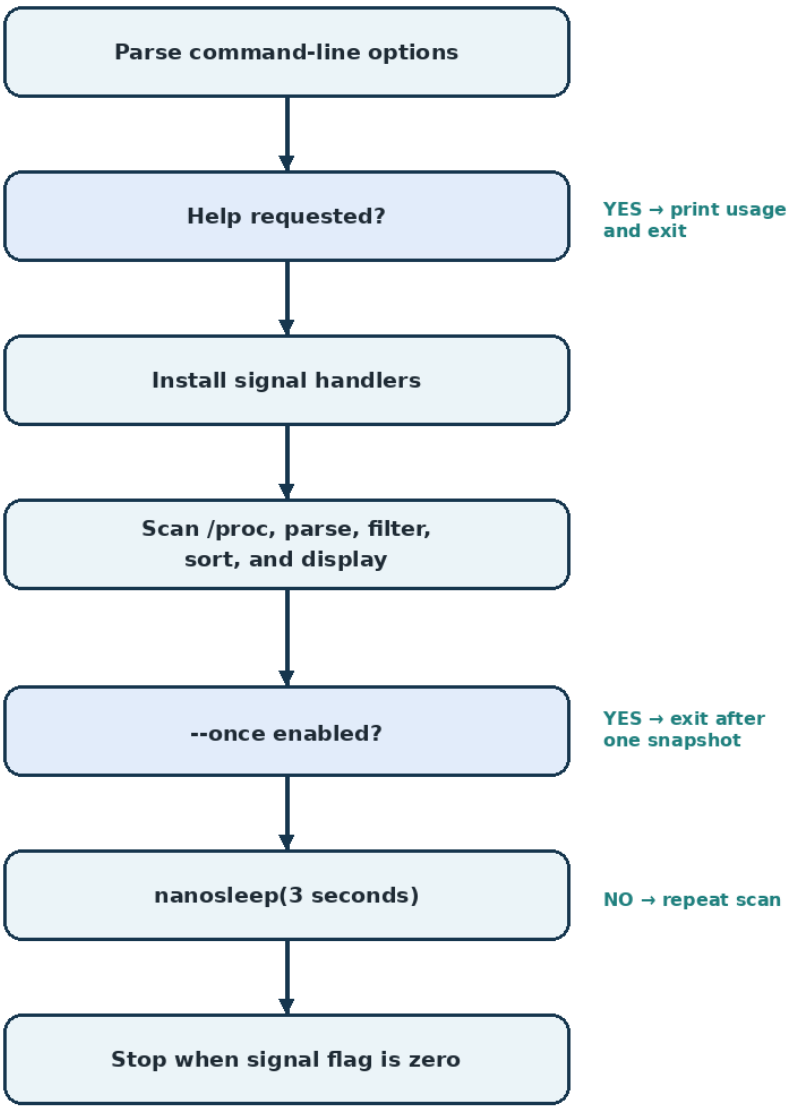


Figure 2. Program flowchart including snapshot and live refresh paths.

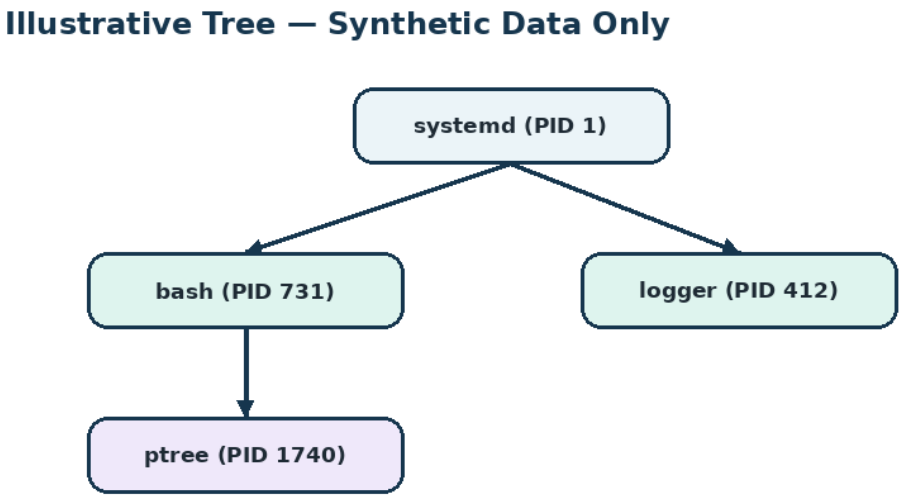


Figure 3. Illustrative process tree using synthetic identifiers.

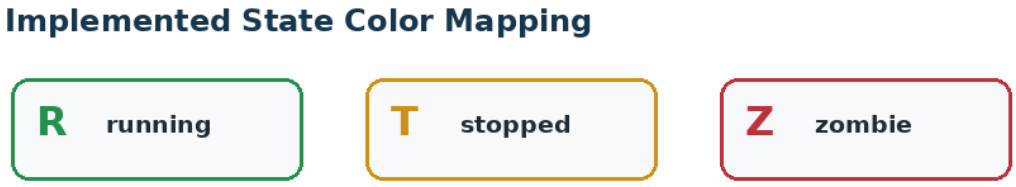


Figure 4. Implemented process-state color mapping.

7.1 Compilation Environment

Save the source as ptree.c in a Linux working directory. Use a terminal with GCC and /proc access. This report does not assert that compilation was executed in the user's environment.

7.2 Source Organization

The minimal runtime project contains ptree.c and the compiled ptree executable. No database, configuration file, network service, or external runtime library is required.

7.3 Compilation Procedure

Run `gcc -Wall -Wextra -O2 -o ptree ptree.c`. A successful build creates ptree. Capture the actual command and terminal result for the submitted report.

7.4 Execution Commands

`./ptree` runs live mode; `./ptree --all` disables the default filter; `./ptree --once` prints one snapshot; `./ptree --help` prints usage. `--interval` is not implemented.

7.5 Command-Line Options

--all includes records that the default PID 2 / PPID 2 filter would skip. --once exits after one scan and display. --help exits after usage text. Unknown options return status 1.

7.6 Sample Terminal Output

```
Illustrative Example — Not Actual Execution Output

LINUX PROCESS TREE VISUALIZER
Refresh interval : 3 seconds
Kernel threads : hidden (use --all)
Processes found : [count from actual run]
Running: [R count] | Stopped: [T count] | Zombies: [Z count]

systemd (PID: 1, PPID: 0, State: S)
└─ bash (PID: 731, PPID: 1, State: S)
   └─ ptree (PID: 1740, PPID: 731, State: R)

Names and identifiers are synthetic; replace them with genuine output before submission.
```

7.7 Output Fields

Name is parsed from stat; PID identifies the record; PPID identifies its recorded parent; state is a one-character code; branch glyphs show matched ancestry. The process total reflects records that survived parsing, filtering, and capacity limits.

7.8 State Demonstration

The implementation colors R green, T yellow, and Z red. A state demonstration should use controlled processes and record the observed terminal output. No state demonstration is claimed as executed here.

7.9 Live Refresh

Run ./ptree in one terminal and create or terminate a harmless child process from another terminal. The next scan may reflect the change. Capture before-and-after terminal screenshots; the snapshot is not atomic.

7.10 Signal Termination

Ctrl+C sends SIGINT to the foreground program. SIGTERM can be sent to the visualizer after confirming its PID. The expected result is that the flag changes and the loop stops. Actual evidence must be recorded locally.

7.11 Test Plan

Test compilation, help, one-shot mode, live refresh, filter behavior, process lifecycle changes, transient read failures, Ctrl+C, SIGTERM, state colors, parent-child indentation, and capacity limits. Record command, environment, observation, exit status, and evidence.

7.12 Test Cases and Expected Results

The table distinguishes expected behavior from actual result. Because no terminal logs were supplied, actual results are marked Not Executed.

7.13 Actual Test Results

No compiler log, execution transcript, or genuine terminal screenshot was provided. Therefore this report does not claim any test passed. Update the matrix after running the program.

7.14 Limitations and Observations

Limitations include Linux /proc dependence, non-atomic snapshots, permission-related failures, a 20,000-record capacity, repeated linear scans, recursion limits, terminal size, ANSI compatibility, fixed refresh interval, and the narrow PID 2 / PPID 2 filter.

ID	Scenario	Procedure	Expected	Actual	Status
T01	Compile	Run GCC command	Build succeeds	Not Executed	Not Executed
T02	Help	./ptree --help	Usage; exit 0	Not Executed	Not Executed
T03	One-shot	./ptree --once	One snapshot	Not Executed	Not Executed
T04	Live	./ptree	Refresh about every 3 seconds	Not Executed	Not Executed
T05	Filter	Compare default and --all	Default skips PID 2/PPID 2 matches	Not Executed	Not Executed
T06	Lifecycle	Create/exit child	Next snapshot may change	Not Executed	Not Executed
T07	Transient failure	Observe race during scan	Unreadable entry skipped	Not Executed	Not Executed
T08	Ctrl+C	Press Ctrl+C	Loop stops	Not Executed	Not Executed
T09	SIGTERM	Signal ptree PID	Loop stops	Not Executed	Not Executed
T10	State colors	Observe R/T/Z where available	R green, T yellow, Z red	Not Executed	Not Executed
T11	Hierarchy	Cross-check PID/PPID	Child under parent	Not Executed	Not Executed
T12	Capacity	Large process collection	No array overflow	Not Executed	Not Executed
T13	Unknown option	./ptree --interval 2	Error and exit 1	Not Executed	Not Executed

Table 7. Test plan; no tests are marked passed without evidence.

SCREENSHOT PLACEHOLDER: Insert a genuine compilation terminal capture here.

SCREENSHOT PLACEHOLDER: Insert genuine ./ptree --once output showing the summary and hierarchy.

SCREENSHOT PLACEHOLDER: Insert genuine live refresh and Ctrl+C termination captures.

CHAPTER - 8 RESULTS, LIMITATIONS, AND FUTURE ENHANCEMENTS

8.1 Functional Results

Source inspection shows code paths for collection, parsing, sorting, tree rendering, state counts, one-shot mode, live refresh, and signal-requested shutdown. This is a description of implementation, not a claim that runtime tests passed.

8.2 Objectives Achieved

The supplied source contains mechanisms for the project's core objectives. Each should be validated locally with the test plan before being reported as experimentally confirmed.

8.3 OS Concepts Demonstrated

The project connects process IDs, parent-child relationships, state codes, /proc metadata, directory/file operations, recursion, signals, and timed waiting.

8.4 Current Limitations

The application depends on Linux /proc; the snapshot is not atomic; some entries may be unreadable; capacity is fixed at 20,000; repeated scans cause $O(n^2)$ worst-case display behavior; recursion is capped at depth 1000; the prefix buffer is fixed; terminal height and ANSI support vary; refresh is fixed at three seconds; and PID 2 / PPID 2 filtering is only a heuristic.

8.5 Future Enhancements

Possible improvements include configurable refresh intervals, process search, filtering, subtree collapsing, a PID hash map or adjacency lists, iterative traversal, no-color output, JSON/text export, explicit truncation warnings, stronger return-value checks, and automated integration tests. These are proposed, not implemented.

8.6 Educational Impact

The source is small enough to inspect while still demonstrating practical Linux interfaces. Students can trace how process metadata becomes a tree and evaluate trade-offs between simple data structures and scalability.

Limitation	Effect	Possible improvement
/proc dependence	Linux-specific	Document environment
Non-atomic snapshot	Transient inconsistencies	Report scan timing
Permission failures	Some records omitted	Count skipped entries
Fixed capacity	20,000 record limit	Dynamic array
Linear scans	Quadratic worst case	PID hash map / adjacency list
Recursion	Depth limit	Iterative traversal
Terminal constraints	Wrapping and scrolling	Paging/collapse

Limitation	Effect	Possible improvement
ANSI colors	Compatibility varies	No-color mode
Fixed interval	No runtime tuning	Add validated --interval
PID 2 filter	Heuristic only	Documented classifier

Table 8. Limitations and future improvements.

Further Design and Validation Notes

Kernel filter caveat

The default filter checks PID == 2 or PPID == 2. It is an implementation-specific heuristic, not a comprehensive classifier for all kernel threads in all Linux configurations.

Capacity and recursion

Records beyond the 20,000-record capacity are skipped. Recursive calls stop beyond depth 1000. These bounds prevent uncontrolled growth but can make an extreme tree incomplete.

Source review

The implementation uses bounded buffers and an explicit name terminator. Some return values are not checked comprehensively. These are review observations, not claims that runtime faults were observed.

Flat list versus tree

A flat list is convenient for sorting and filtering, while a tree highlights ancestry. This application prioritizes hierarchy over CPU, memory, elapsed time, or interactive controls.

Safe lab workflow

Compile, run --help, run --once, then run live mode. Use a harmless child process in a second terminal to observe refresh, then stop with Ctrl+C. Do not signal unrelated system services.

Future automated testing

A test harness could create a controlled parent-child chain and compare observed PIDs and PPIDs. Tests should not rely on fixed host PIDs or names because they vary between runs.

Interpreting roots

A displayed root is a record whose parent is missing from the collected set or whose PPID is non-positive. The parent may be filtered, inaccessible, outside the visible namespace, or gone during the scan.

Memory and buffer safety

Fixed bounds apply to the record array, name, stat line, path, and prefix. The capacity guard prevents array overflow, but fixed buffers can truncate unusual values. Better diagnostics could report skipped records.

Ethical testing

Use processes created by the student and avoid terminating unrelated services. Confirm the visualizer PID before sending SIGTERM. Avoid uncontrolled process storms on shared lab machines.

CHAPTER - 9 CONCLUSION

9.1 Project Summary

Linux Process Tree Visualizer reads process metadata from /proc/<pid>/stat and renders PID/PPID relationships as a terminal tree. It provides selected state colors, R/T/Z counts, one-shot mode, live refresh, command-line help, and signal-requested shutdown.

9.2 Objectives Achieved

The supplied source implements numeric directory scanning, metadata parsing, PID sorting, root detection, recursive child rendering, state highlighting, and refresh control. Runtime test success is not asserted without evidence.

9.3 Technical Learning Outcomes

The project provides practice with C structures and arrays, bounded buffers, directory traversal, file reading, string parsing, qsort(), linear search, recursion, ANSI sequences, POSIX signals, and nanosleep().

9.4 OS Concepts Demonstrated

The key concepts are PIDs, PPIDs, process hierarchy, process states, Linux /proc, signals, and best-effort observation of a changing system.

9.5 Final Conclusion

The project connects operating-system theory with a compact systems-programming design. The hierarchy is created by matching PPID to PID; sorting only makes the output order predictable. The implementation is educational but has documented scaling and display limits. Genuine local compilation and testing evidence should be added before submission.

CHAPTER - 10 REFERENCES AND APPENDICES

10.1 References

[1] Linux man-pages project, “proc(5) — process information pseudo-filesystem,” https://man7.org/linux/man-pages/man5/proc.5.html
[2] Linux man-pages project, “proc_pid_stat(5) — process status information,” https://man7.org/linux/man-pages/man5/proc_pid_stat.5.html
[3] Linux man-pages project, “sigaction(2),” https://man7.org/linux/man-pages/man2/sigaction.2.html
[4] Linux man-pages project, “nanosleep(2),” https://man7.org/linux/man-pages/man2/nanosleep.2.html
[5] GNU Project, GCC documentation, https://gcc.gnu.org/onlinedocs/
[6] The Open Group, POSIX.1-2017 Base Specifications, https://pubs.opengroup.org/onlinepubs/9699919799/
[7] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts. Verify the library edition before adding edition-specific details.

10.2 Compilation Quick Reference

Compile: gcc -Wall -Wextra -O2 -o ptree ptree.c
Live mode: ./ptree
Include filtered records: ./ptree --all
Single snapshot: ./ptree --once
Usage: ./ptree --help
Stop live mode: Ctrl+C or SIGTERM.
The --interval option is not supported.

10.3 Additional Test Cases

Recommended additional checks: combine --all --once; pass an unknown option; compare a known child PPID to the parent PID; inspect behavior on a terminal without ANSI support; run with many processes; and verify the depth guard in a controlled harness. Mark results only after observing them.

10.4 Glossary

Process: executing program and operating-system context. PID: process identifier. PPID: parent process identifier. /proc: Linux pseudo-filesystem for process and kernel information. Snapshot: one collection/display cycle. Zombie: terminated process whose exit status has not yet been fully reaped. Signal: asynchronous process notification. Recursion: a function calling itself. Time complexity: how work grows with input size.

10.5 Screenshot Placeholders

Insert genuine terminal captures after execution: (1) successful compilation; (2) --help output; (3) --once output with hierarchy and state summary; (4) live refresh; (5) Ctrl+C termination; (6) --all mode; (7) a child whose PPID matches a displayed parent PID. Do not substitute mock terminal images for execution evidence.

10.6 Complete Supplied C Source Code

The source listing below is reproduced from the supplied implementation for traceability.

Appendix E — Complete Supplied C Source Code

```
/*
 * Linux Process Tree Visualizer
 * -----------------------------
 * Reads /proc, builds the parent-child hierarchy of running processes
 * and displays it as a colored tree that refreshes in real time.
 *
 * Compile : gcc -Wall -Wextra -O2 -o ptree ptree.c
 * Run     : ./ptree            live view (kernel threads hidden)
 *           ./ptree --all      include kernel threads
 *           ./ptree --once     print one snapshot and exit
 *           ./ptree --help     show usage
 */
#define _POSIX_C_SOURCE 200809L
#include <ctype.h>
#include <dirent.h>
#include <signal.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <unistd.h>
#define MAX_PROCESSES   20000
#define NAME_SIZE       256
#define REFRESH_SECONDS 3
#define COLOR_RESET  "\033[0m"
#define COLOR_GREEN  "\033[32m"
#define COLOR_YELLOW "\033[33m"
#define COLOR_RED    "\033[31m"
typedef struct {
    int  pid;
    int  ppid;
    char name[NAME_SIZE];
    char state;
    bool printed;
} Process;
static Process processes[MAX_PROCESSES];
static int process_count = 0;
static volatile sig_atomic_t running = 1;
static bool once = false;
static bool show_kernel = false;
static void handle_signal(int signal_number)
{
    (void)signal_number;
    running = 0;
}
static int compare_processes(const void *a, const void *b)
{
    const Process *x = a;
    const Process *y = b;
    if (x->pid < y->pid) return -1;
    if (x->pid > y->pid) return 1;
    return 0;
}
static int find_process(int pid)
{
    for (int i = 0; i < process_count; i++) {
        if (processes[i].pid == pid)
            return i;
    }
    return -1;
}
static const char *state_color(char state)
{
    switch (state) {
        case 'R': return COLOR_GREEN;
        case 'T': return COLOR_YELLOW;
        case 'Z': return COLOR_RED;
        default:  return COLOR_RESET;
    }
}
static void print_process(const Process *p)
{
    printf("%s (PID: %d, PPID: %d, State: %s%c%s)\n",
           p->name, p->pid, p->ppid,
           state_color(p->state), p->state, COLOR_RESET);
}
static bool read_process(const char *directory_name, Process *p)
```

```
{
    char path[64];
    char buffer[4096];
    snprintf(path, sizeof(path), "/proc/%s/stat", directory_name);
    FILE *file = fopen(path, "r");
    if (file == NULL)
        return false;
    if (fgets(buffer, sizeof(buffer), file) == NULL) {
        fclose(file);
        return false;
    }
    fclose(file);
    char *left  = strchr(buffer, '(');
    char *right = strrchr(buffer, ')');
    if (left == NULL || right == NULL || right <= left)
        return false;
    long pid = strtol(directory_name, NULL, 10);
    if (pid <= 0 || pid > 2147483647L)
        return false;
    size_t length = (size_t)(right - left - 1);
    if (length >= sizeof(p->name))
        length = sizeof(p->name) - 1;
    memcpy(p->name, left + 1, length);
    p->name[length] = '\0';
    if (sscanf(right + 1, " %c %d", &p->state, &p->ppid) != 2)
        return false;
    p->pid = (int)pid;
    p->printed = false;
    return true;
}
static void scan_processes(void)
{
    process_count = 0;
    DIR *proc = opendir("/proc");
    if (proc == NULL) {
        perror("Cannot open /proc");
        return;
    }
    struct dirent *entry;
    while ((entry = readdir(proc)) != NULL) {
        bool numeric = isdigit((unsigned char)entry->d_name[0]);
        for (const char *s = entry->d_name; numeric && *s; s++) {
            if (!isdigit((unsigned char)*s))
                numeric = false;
        }
        if (!numeric || process_count >= MAX_PROCESSES)
            continue;
        Process p;
        if (!read_process(entry->d_name, &p))
            continue;
        if (!show_kernel && (p.pid == 2 || p.ppid == 2))
            continue;
        processes[process_count++] = p;
    }
    closedir(proc);
    qsort(processes, process_count, sizeof(Process), compare_processes);
}
static int count_children(int pid)
{
    int count = 0;
    for (int i = 0; i < process_count; i++) {
        if (processes[i].ppid == pid && !processes[i].printed)
            count++;
    }
    return count;
}
static void print_node(int index, const char *prefix,
                       bool is_last, bool is_root, int depth)
{
    if (depth > 1000 || processes[index].printed)
        return;
    processes[index].printed = true;
    if (is_root) {
        print_process(&processes[index]);
    } else {
        printf("%s%s ", prefix,
               is_last ? "\u2514\u2500\u2500" : "\u251C\u2500\u2500");
        print_process(&processes[index]);
    }
    int remaining = count_children(processes[index].pid);
    char next_prefix[4096];
    if (is_root) {
        next_prefix[0] = '\0';
    } else {
        snprintf(next_prefix, sizeof(next_prefix), "%s%s", prefix,
                 is_last ? "    " : "\u2502   ");
    }
    for (int i = 0; i < process_count; i++) {
```

```
        if (processes[i].ppid != processes[index].pid ||
            processes[i].printed)
            continue;
        remaining--;
        print_node(i, next_prefix, remaining == 0, false, depth + 1);
    }
}
static void display_tree(void)
{
    int zombies = 0, stopped = 0, running_count = 0;
    for (int i = 0; i < process_count; i++) {
        if (processes[i].state == 'Z') zombies++;
        if (processes[i].state == 'T') stopped++;
        if (processes[i].state == 'R') running_count++;
    }
    if (!once)
        printf("\033[2J\033[H");
    printf("==========================================\n");
    printf("       LINUX PROCESS TREE VISUALIZER\n");
    printf("==========================================\n");
    if (once)
        printf("Mode             : single snapshot\n");
    else
        printf("Refresh interval : %d seconds\n", REFRESH_SECONDS);
    printf("Kernel threads   : %s\n", show_kernel ? "shown" : "hidden (use --all)");
    printf("Processes found  : %d\n", process_count);
    printf("Running: %d | Stopped: %d | Zombies: %d\n",
           running_count, stopped, zombies);
    printf("Legend: %sR=running%s  %sT=stopped%s  %sZ=zombie%s\n",
           COLOR_GREEN, COLOR_RESET, COLOR_YELLOW, COLOR_RESET, COLOR_RED, COLOR_RESET);
    printf("==========================================\n\n");
    for (int i = 0; i < process_count; i++) {
        if (processes[i].printed)
            continue;
        if (processes[i].ppid <= 0 || find_process(processes[i].ppid) == -1)
            print_node(i, "", true, true, 0);
    }
    for (int i = 0; i < process_count; i++) {
        if (!processes[i].printed)
            print_node(i, "", true, true, 0);
    }
    if (!once)
        printf("\nPress Ctrl+C to exit.\n");
    fflush(stdout);
}
static void print_usage(const char *program)
{
    printf("Usage: %s [--all] [--once] [--help]\n", program);
    printf("  --all   include kernel threads\n");
    printf("  --once  print one snapshot and exit\n");
    printf("  --help  show this message\n");
}
int main(int argc, char **argv)
{
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "--once") == 0) {
            once = true;
        } else if (strcmp(argv[i], "--all") == 0) {
            show_kernel = true;
        } else if (strcmp(argv[i], "--help") == 0) {
            print_usage(argv[0]);
            return 0;
        } else {
            fprintf(stderr, "Unknown option: %s\n", argv[i]);
            print_usage(argv[0]);
            return 1;
        }
    }
    struct sigaction action = {0};
    action.sa_handler = handle_signal;
    sigemptyset(&action.sa_mask);
    sigaction(SIGINT, &action, NULL);
    sigaction(SIGTERM, &action, NULL);
    while (running) {
        scan_processes();
        display_tree();
        if (once)
            break;
        struct timespec delay = { .tv_sec = REFRESH_SECONDS, .tv_nsec = 0 };
        while (running && nanosleep(&delay, &delay) == -1) {
        }
    }
    if (!once)
        printf("\nProcess Tree Visualizer stopped.\n");
    return 0;
}
```

Appendix — Detailed Command-Line Interface

The command-line interface is intentionally small. The program accepts three optional flags and rejects every unrecognized argument. This makes invocation predictable and prevents silently ignoring a spelling mistake.

The default invocation starts a live view. The --once flag changes the loop into a single scan and display. The --all flag changes the filtering rule for process records. The --help flag prints usage information and exits before the monitoring loop begins.

Flags can be combined because main() iterates through every argument. For example, ./ptree --all --once requests one snapshot without applying the default PID-2 filter. The order of these flags does not change their meaning.

Invocation	**Expected behaviour**	**Evidence to capture**
./ptree	Refreshes view every three seconds	Terminal view
./ptree --once	Prints one snapshot then exits	Shell prompt returns
./ptree --all	Includes records normally filtered by PID/PPID 2	Compare output
./ptree --all --once	One snapshot with filter disabled	One-shot output
./ptree --help	Prints usage and exits	Usage text
./ptree --badflag	Error and usage; non-zero status	Error text

Appendix — Argument Parsing and Program Modes

Argument parsing occurs before the main monitoring loop. Each argument is compared with a known literal using strcmp(). A recognized option updates a Boolean mode variable; an unknown option is reported on standard error and terminates execution with status 1.

The --help branch returns immediately with status 0. This is an informational request, not an execution failure. The mode variables are global static Booleans initialized to false, so the default mode is a live view with the default process filter enabled.

The running flag begins true and controls the outer loop. The once flag controls whether the loop exits after the first displayed snapshot.

Variable	Initial value	Meaning
once	false	Print only one snapshot
show_kernel	false	Disable the PID/PPID 2 filter
running	1	Whether the live loop continues
process_count	0	Accepted process records

Appendix — Linux procfs Overview

/proc is a virtual filesystem exposed by the Linux kernel. Its entries provide process and kernel information through file-like interfaces; they are not ordinary persistent files stored on disk. The visualizer opens /proc and examines directory names to discover numeric process identifiers.

Each numeric directory commonly contains a stat file. The implementation reads /proc/<pid>/stat to obtain process name, state and parent PID. Since processes can terminate between directory enumeration and file opening, a missing or unreadable stat file is treated as a record that cannot be used for this scan.

procfs contents are a view of changing kernel state rather than a transactionally consistent database. Parent and child information may change during collection, so the output is a best-effort live view.

procfs element	**Role in the program**	**Important note**
/proc	Directory enumerated by scan_processes()	Can fail to open
/proc/<pid>/stat	Source of PID-related metadata	May disappear during scan
Numeric directory name	Candidate process ID	Non-numeric entries skipped
State field	R, T, Z or another state	Only three are specially coloured

Appendix — Parsing /proc/[pid]/stat

The stat record begins with a process identifier and a parenthesized command name, followed by state and additional fields. The command name can contain spaces and parentheses, so splitting the whole line on spaces would be unsafe.

The implementation locates the first opening parenthesis and the last closing parenthesis. The substring between them is copied into the fixed-size name field. The length is capped at NAME_SIZE minus one so a null terminator can be stored.

After the closing parenthesis, sscanf() reads the state character and parent PID. The parser checks that two conversions succeeded before accepting the record. Malformed or incomplete records are rejected.

Parsing step	Implementation technique	Reason
Find command boundaries	strchr() and strrchr()	Name may contain spaces
Copy process name	memcpy() with bounded length	Avoid overflowing name field
Terminate name	Set p->name[length] to '\0'	Maintain valid C string
Read state and PPID	sscanf() after closing parenthesis	Avoid parsing name as fields
Validate PID	strtol() and range checks	Reject invalid identifiers

Appendix — Bounds, Buffers and Defensive Programming

The program uses bounded arrays and buffers: the process table can hold MAX_PROCESSES records, process names use NAME_SIZE bytes, the stat line uses a 4096-byte buffer, and constructed file paths use a 64-byte buffer. These bounds make memory use predictable but create explicit limits.

Before adding a record, scan_processes() checks that process_count has not reached MAX_PROCESSES. Names longer than the destination capacity are truncated to fit. The final byte is set to the null terminator, preserving valid C-string semantics.

The program does not dynamically allocate one object per process. This avoids per-record allocation and cleanup complexity, but a future revision could use dynamic storage for larger process populations.

Resource	**Defined limit**	**Handling**
Process table	20,000 records	Stop accepting additional records
Process name	255 characters plus terminator	Truncate longer names
Stat input buffer	4096 bytes	Reject failed or incomplete read
Path buffer	64 bytes	Construct /proc/<pid>/stat
Tree prefix	4096 bytes	Build indentation for descendants

Appendix — Process State Semantics

The process state character is obtained from the stat record and stored without translating it into a longer label. The display uses a colour function for three states that are useful in a demonstration: R, T and Z.

R denotes a running or runnable task; T denotes a stopped or traced state; Z denotes a zombie process that has terminated but remains represented until its exit status is collected. The program does not claim that every process must fall into one of these three highlighted states.

Other state characters are printed with the terminal's default colour. Colour is a visual aid, not a diagnosis of process health. A zombie is not an actively executing process.

State	Colour in this program	Interpretation
R	Green	Running or runnable
T	Yellow	Stopped or traced
Z	Red	Zombie
Other	Default	Not specially highlighted

Appendix — Tree Representation and Connectors

The visualizer renders a parent-child hierarchy with Unicode line-drawing characters. A non-final child uses a vertical continuation line and a branch connector. The final child uses a corner connector, making the hierarchy easier to follow.

The prefix string represents the vertical context inherited from ancestors. If an ancestor has more siblings after the current branch, the next prefix retains a vertical line. If the current branch is the final sibling, the next prefix uses spaces instead.

Terminal fonts and locale settings affect how Unicode characters appear. If the terminal cannot render these characters, the process data may still be correct while connectors appear as replacement symbols.

Connector	**Purpose**
├──	Shows a non-final child
└──	Shows the final child
│	Continues a vertical ancestor branch
Spaces	Keeps indentation where no branch continues

Appendix — Root Detection

A tree needs starting nodes. The display routine considers a record a root when its PPID is non-positive or when no process with that parent PID exists in the collected table. This allows display even when the parent process is outside the captured set.

The find_process() helper searches the current array for a matching PID. If no match exists, the process is printed as a root. This can happen when a parent exits during collection, when a process is hidden by the selected filter, or when a process is not visible to the current user.

After roots are traversed, a safety pass prints any records still unprinted. This prevents the display from omitting remaining records if inconsistent relationships prevent normal root traversal.

Condition	**Effect**
PPID <= 0	Treat record as a root
Parent PID not found	Treat record as a root of the collected subset
Parent excluded by filter	Child may appear disconnected
Record not reached by root walk	Fallback pass attempts to print it

Appendix — Recursive Traversal

print_node() is a recursive depth-first traversal. It marks the current process as printed before visiting children, preventing repeated traversal of the same record. It then counts eligible children and recursively displays them in array order.

The recursion depth is limited by a guard at 1000. This avoids unbounded recursion if malformed relationships are encountered, but a sufficiently deep legitimate hierarchy could also be cut short. The bound is a protective implementation decision.

The array is sorted by PID before traversal. Siblings are therefore considered in ascending PID order, making successive displays more predictable and easier to compare.

Stage	Action	Purpose
1	Check depth and printed flag	Stop unsafe or duplicate traversal
2	Mark node printed	Avoid revisiting the same record
3	Print current process	Show name, PID, PPID and state
4	Count eligible children	Determine final connector
5	Build descendant prefix	Maintain indentation
6	Visit matching children	Produce depth-first hierarchy

Appendix — Sorting and Lookup Complexity

qsort() sorts accepted process records by PID. The comparator returns a negative value, zero, or a positive value depending on whether the first PID is smaller, equal, or greater. Sorting makes sibling traversal predictable for a given collected set.

find_process() uses a linear scan, so one lookup costs O(n) in the worst case. Root detection may invoke this search for many records. count_children() also scans the full array, and traversal scans the array to find children.

For n accepted processes, sorting is typically O(n log n). Several tree-building operations can take $O(n^2)$ in the worst case because they repeatedly scan the process array. This favours straightforward code over optimal asymptotic performance.

Operation	**Approximate cost**	**Reason**
Sort records	O(n log n) typical	qsort() on n records
find_process(pid)	O(n)	Linear search
count_children(pid)	O(n)	Full-array scan
Find children during traversal	Repeated O(n) scans	Each node scans records
Overall rendering	Can approach $O(n^2)$	Repeated full-array scans

Appendix — Signal Handling and Clean Shutdown

The program installs handlers for SIGINT and SIGTERM using sigaction(). The handler does only one operation: it sets the volatile sig_atomic_t variable running to zero. This is a minimal signal handler and avoids formatted output or complex work in asynchronous signal context.

The live loop checks running before another scan and while waiting. When nanosleep() is interrupted, the inner loop attempts to continue sleeping with the remaining duration while the program is still meant to run. After leaving the loop, the program prints a final stopped message in live mode.

SIGINT commonly corresponds to Ctrl+C in a terminal. SIGTERM is a conventional termination request. The one-shot mode exits after displaying the first snapshot.

Element	**Purpose**
sigaction()	Registers signal handlers
handle_signal()	Sets running to zero
sig_atomic_t	Suitable simple signal flag
nanosleep() loop	Waits while permitting shutdown
Final message	Printed after live loop ends

Appendix — Refresh Cycle and Timing

The live mode repeats three steps: scan /proc, display the tree, then wait for the refresh interval. REFRESH_SECONDS is set to 3. The display clears the terminal using ANSI escape sequences before printing the latest summary and tree.

The refresh period is a target delay between cycles rather than a strict real-time guarantee. Scanning and rendering take time, so elapsed time from one snapshot start to the next is approximately the scan-and-render duration plus the sleep duration.

nanosleep() can be interrupted by a signal. The code loops while running and nanosleep returns -1, passing the remaining time back to nanosleep. Once running becomes zero, the wait loop ends.

Cycle stage	**Function**	**Result**
Collect	scan_processes()	Accepted records
Sort	qsort() inside scan	PID-ordered records
Summarize and render	display_tree()	Header, counts and hierarchy
Wait	nanosleep()	Three-second delay target
Repeat or exit	running / once	Live refresh or termination

Appendix — ANSI Terminal Colour Support

The program uses ANSI escape sequences for screen clearing and state colouring. The reset sequence returns the terminal to its default attributes. The three state colours are green for R, yellow for T and red for Z.

ANSI escape sequences are interpreted by most modern terminal emulators, but they are not ordinary printable characters. If output is redirected to a file, the file may contain escape codes. A terminal without ANSI support may display the codes literally or ignore them.

The live view clears the screen with a clear-screen and cursor-home sequence. A possible enhancement is a --no-color option for logs and automated testing.

Constant	**Use**
COLOR_RESET	Reset terminal formatting
COLOR_GREEN	Highlight R state
COLOR_YELLOW	Highlight T state
COLOR_RED	Highlight Z state
Clear-screen sequence	Refresh dashboard without scrolling

Appendix — Error Handling Review

The code handles several ordinary failures by skipping a process record or returning an error status. Failure to open a process stat file is expected when a process exits during the scan. Failure to parse the expected fields also causes the record to be skipped.

If opendir("/proc") fails, perror() reports the system error and scan_processes() returns. The main loop then displays the count, which is zero after process_count is reset. In live mode, it retries after the next refresh.

Some system calls have return values that the supplied code does not inspect, including sigaction(). This is a specific improvement opportunity. A more robust implementation could report failure to install handlers.

Failure	Current response	Possible enhancement
Cannot open /proc	perror and return from scan	Return status to main
Cannot open process stat	Skip that process	Optional debug logging
Malformed stat line	Reject record	Count parse failures
Unknown CLI option	Error and usage; exit 1	Already handled
Signal setup failure	Return value not checked	Check sigaction() result
Capacity reached	Stop accepting records	Report truncation explicitly

Appendix — Security and Privilege Considerations

The program is a read-only observer of process metadata. It does not send signals to other processes, change scheduling priority, write to process memory, or alter kernel state. Its primary interaction is opening procfs entries and printing collected information.

Process visibility depends on operating-system configuration, permissions, namespaces and the privileges of the executing user. Running as a more privileged user can reveal additional records, but privilege escalation is not required for a basic demonstration.

Directory names are discovered from /proc rather than supplied directly by a user. The implementation checks that names consist entirely of digits before constructing a stat path, limiting path construction to the intended procfs entry pattern.

Good practice	**Reason**
Use least privilege	Avoid unnecessary elevated execution
Do not change /proc permissions	Not required for normal use
Interpret missing records cautiously	Visibility and process churn affect results
Keep the tool read-only	Matches the current implementation scope

Appendix — Functional Test Plan

The following test plan defines checks that can be performed on a Linux host. These are proposed test cases, not claims that tests have already been executed. A test record should include the environment, command, expected behaviour, actual result, date and screenshot or log.

Tests should cover normal execution, each supported option, unknown arguments, live refresh, termination and state-colour rendering when suitable processes are present. State-specific tests depend on the host actually having a process in the relevant state.

Because process information changes continuously, exact process counts should not be hard-coded as expected results. Better checks are structural: output contains the header, process records have PID/PPID/state fields, and the requested mode exits or refreshes as expected.

ID	Test	Expected result	Status
TC-01	Run --help	Usage displayed; exits successfully	Not executed
TC-02	Run --once	One snapshot then exit	Not executed
TC-03	Run default mode	View refreshes repeatedly	Not executed
TC-04	Press Ctrl+C	Loop stops and final message appears	Not executed
TC-05	Pass unknown flag	Error and usage; non-zero exit	Not executed
TC-06	Run --all --once	One snapshot with filter disabled	Not executed
TC-07	Inspect state colours	R/T/Z highlighted if present	Not executed
TC-08	Test unavailable /proc	Error path is reported	Not executed

Appendix — Non-Functional Evaluation Criteria

Non-functional evaluation considers how the program behaves beyond whether it produces a tree. Relevant criteria include readability, responsiveness, resource use, portability within Linux environments, resilience to process churn and maintainability.

Responsiveness is affected by process count, repeated linear scans, terminal output volume and the three-second refresh interval. Memory use is bounded by static arrays and fixed-size buffers, which makes maximum storage predictable.

Maintainability benefits from helper functions with focused responsibilities: parsing, scanning, sorting, lookup, state colouring, printing, tree traversal and usage display.

Criterion	**How to assess**	**Current design note**
Correctness	Compare PPID relationships	Snapshot can change during collection
Responsiveness	Observe larger process populations	Repeated scans can be $O(n^2)$
Memory	Review arrays and buffers	Bounded static storage
Readability	Review indentation and summary	Compact terminal tree
Portability	Build on Linux systems	Requires procfs and POSIX APIs
Maintainability	Inspect helper boundaries	Functions separated by task

Appendix — Portability and Environment

The source depends on Linux procfs, POSIX signal handling, directory iteration and nanosleep(). It is not a generic cross-platform process viewer: operating systems without Linux-style /proc/<pid>/stat entries cannot provide the data in the expected format.

The compile command uses GCC options -Wall and -Wextra to enable warnings and -O2 for optimization. The source defines _POSIX_C_SOURCE 200809L before headers so POSIX declarations such as nanosleep() and sigaction() are available on conforming systems.

The terminal output uses Unicode line connectors and ANSI control sequences. For the best display, use a UTF-8-capable terminal emulator that supports ANSI escape sequences.

Command	**Purpose**
gcc -Wall -Wextra -O2 -o ptree ptree.c	Compile with warnings and optimization
./ptree --once	Run a one-shot snapshot
./ptree --all --once	One-shot snapshot with filter disabled
./ptree --help	Display supported options

Appendix — Build and Execution Procedure

Save the implementation in a file named ptree.c on a Linux system with a C compiler installed. Compile it from the directory containing the file using the command documented in the source header. The executable is named ptree.

After compilation, begin with --help to verify the available flags. Use --once for a compact test that returns control to the shell. Use the default command to observe live refresh, and press Ctrl+C to stop it. Keep a record of the environment and compiler output for reproducibility.

Warnings should be reviewed rather than automatically ignored. A successful compile confirms that the source was accepted by the compiler, but it does not by itself prove every runtime behaviour.

Step	Action
1	Open a Linux terminal
2	Save source as ptree.c
3	Compile using gcc -Wall -Wextra -O2 -o ptree ptree.c
4	Run ./ptree --help
5	Run ./ptree --once and inspect output
6	Run ./ptree and stop with Ctrl+C
7	Record outcomes and capture genuine screenshots

Appendix — Sample Output Interpretation

The exact process names, identifiers and counts depend on the machine and time of execution. The header provides the project name, refresh mode, kernel-thread filter setting, number of accepted records, and counts of R, T and Z states.

Each process line contains the command name, PID, PPID and state. The connectors indicate the relationship inferred from PPID values among records included in the current scan. A displayed root is not necessarily the only root in the operating system; it may be a root of the collected subset.

Sample output should be produced on the target machine rather than invented for the report. A screenshot should show the terminal command and enough output to explain the feature.

Output field	Meaning	Caution
Processes found	Accepted records in this scan	May exclude inaccessible or filtered processes
Running count	Records with state R	Not CPU utilization
Stopped count	Records with state T	Only collected records
Zombie count	Records with state Z	Zombie records are not executing
PID / PPID	Process and parent identifiers	May change between refreshes
Tree connectors	Inferred parent-child relation	Snapshot is not atomic

Appendix — Limitations and Threats to Validity

The process table is limited to 20,000 records. If a host exposes more eligible entries, the scan stops adding records after reaching capacity. The current display does not explicitly state that truncation occurred, so very large-host output should be treated cautiously.

The collection is not atomic. A process can start, exit, change parentage or become inaccessible while scanning. A child can be collected while its parent is missing from the accepted array, causing it to appear as a root.

Repeated linear scans can become expensive as the process count grows. The recursion guard limits depth, and terminal output can become very large. This is an educational visualizer rather than a replacement for production process-monitoring utilities.

Limitation	**Potential effect**	**Mitigation direction**
Fixed capacity	Records beyond limit omitted	Dynamic allocation and truncation notice
Non-atomic scan	Inconsistent parent-child snapshot	Document best-effort semantics
Linear lookup	Higher cost on large sets	PID-to-index hash table
Recursive rendering	Deep tree limited by guard	Iterative traversal
Fixed refresh interval	No user-configurable rate	Add validated interval option
Terminal dependency	Colours may not render	No-colour and ASCII modes

Appendix — Future Enhancement: PID Indexing

A PID-to-index table could reduce the cost of locating a parent. After collecting and sorting records, a hash table or other index can map each PID to its array position. Root detection would then avoid scanning every record for every parent lookup.

Any index must be built from accepted process records and must handle missing parents correctly. If the index is a hash table, collision resolution and load factor become design considerations. The current source does not implement this index; it is a proposed enhancement.

Another design is to build a child adjacency list in one pass. Each process record can be associated with its parent's child list, after which rendering traverses only the children of the current node.

Approach	**Benefit**	**Trade-off**
PID hash index	Fast average parent lookup	Needs collision handling
Child adjacency lists	Efficient child traversal	More memory management
Sorted binary search	O(log n) lookup after sorting	Requires sorted data
Iterative traversal	Avoids deep recursive stack	Explicit stack management

Appendix — Future Enhancement: Configuration and Output Modes

The current refresh interval is a compile-time constant. A future version could accept a validated --interval value while retaining three seconds as the default. The parser should reject zero, negative, malformed or unreasonably large values.

A --no-color option could disable ANSI styling for log files and terminals without colour support. An --ascii option could replace Unicode connectors with plain-text symbols. These features improve compatibility without changing the process model.

Structured output such as CSV or JSON could support downstream analysis, but it should be implemented as a distinct output mode rather than mixing escape codes and tree glyphs into machine-readable data.

Enhancement	**Purpose**	**Status**
Configurable interval	Adjust refresh frequency	Proposed, not implemented
--no-color	Plain output for logs	Proposed, not implemented
--ascii	Restricted-terminal compatibility	Proposed, not implemented
JSON/CSV export	Downstream processing	Proposed, not implemented
Capacity warning	Make truncation visible	Proposed, not implemented

Appendix — Viva Preparation: Core Questions

1. What is procfs? It is a virtual filesystem through which Linux exposes kernel and process information. 2. What are PID and PPID? PID identifies a process; PPID identifies its parent. 3. Why is /proc/<pid>/stat parsed carefully? The command name is parenthesized and can contain spaces or parentheses.

4. Why use qsort()? It arranges records by PID so sibling traversal has a predictable order. 5. Why use recursion? A process hierarchy is naturally represented as a tree. 6. Why mark a node printed? To avoid duplicate traversal and identify unvisited records.

7. Why use sig_atomic_t? It is appropriate for a simple flag shared with a signal handler. 8. Why is the snapshot best-effort? Process state can change during enumeration. 9. What is --once for? It prints a single snapshot and exits. 10. What is the main performance limitation? Repeated full-array scans can approach quadratic work.

Appendix — Viva Preparation: Implementation Details

11. Why does read_process() return bool? It reports whether a usable record could be parsed. 12. Why skip a process if fopen() fails? It may have exited or become inaccessible during scanning. 13. Why use strtol()? It converts the directory name to a numeric PID and permits validation.

14. What is the purpose of the final unprinted-node pass? It is a safety net for records not reached through normal root detection. 15. Why check depth greater than 1000? It limits recursion in abnormal or extremely deep relationships. 16. Why separate state colouring into a helper? It keeps display decisions out of the parser.

17. What happens with an unknown command-line option? The program prints an error and usage and returns status 1. 18. Does the program modify processes? No; it reads metadata and prints it. 19. Why might a child appear as a root? Its parent may be missing from the collected subset. 20. What must happen before claiming tests passed? Run the tests and preserve genuine evidence.

Appendix — Traceability Matrix

This matrix links user-visible features to implementation functions and practical verification. It makes review and viva preparation easier by connecting descriptions to concrete source areas.

Traceability does not imply that tests have already passed. The verification column describes a check to perform on the executable. Status remains unexecuted until an actual run is documented.

Feature	Implementation element	Verification method
Argument flags	main(), print_usage()	Run each supported option
Process discovery	scan_processes()	Inspect accepted records
Stat parsing	read_process()	Compare fields with procfs
PID ordering	compare_processes(), qsort()	Observe ordered records
State colour	state_color(), print_process()	Inspect R/T/Z if present
Tree traversal	print_node()	Compare PPID hierarchy
Root detection	display_tree(), find_process()	Inspect missing-parent cases
Live refresh	main(), nanosleep()	Observe repeated updates
Signal shutdown	handle_signal()	Press Ctrl+C
Summary counts	display_tree()	Compare counts with states

Appendix — Reproducibility Record Template

Reproducibility matters because process data is specific to the host and time of execution. Each demonstration should record enough context for another student or evaluator to understand how the output was obtained.

Complete this record after a genuine run. Do not populate it with guessed values. If a field is not available, state that it was not recorded. Screenshots should be original captures of the running program, not recreated mock-ups.

Field	**Record after testing**
Date and time	_____
Linux distribution and version	_____
Kernel version	_____
Compiler and version	_____
Exact compile command	_____
Exact run command	_____
Exit status	_____
Warnings or errors	_____
Observed process count	_____
Screenshot/log filename	_____
Tester initials	_____

Appendix — Maintenance Checklist

A maintenance checklist helps prevent changes from breaking behaviour that is easy to overlook. After modifying parsing, traversal, command-line handling or formatting, compile with warnings enabled and rerun the functional test plan.

Changes to the process record structure should be reviewed alongside every function that reads, sorts, searches, prints or marks records. Changes to the command-line interface should update print_usage() and the report's feature description. Changes to state handling should update both the legend and test expectations.

Documentation should be updated when a feature is implemented, removed or changes semantics. Do not describe a planned feature as implemented until source code and tests support that statement.

Review item	Done
Compile with -Wall -Wextra	[ ]
Check all CLI modes and unknown options	[ ]
Review buffer sizes and null termination	[ ]
Check signal handler and sleep loop	[ ]
Verify root and fallback traversal	[ ]
Refresh screenshots after output changes	[ ]
Update limitations and traceability	[ ]

Appendix — Project Scope and Educational Outcomes

The project demonstrates how an operating system exposes process metadata and how a user-space program can transform that metadata into a readable hierarchy. It combines file operations, parsing, sorting, recursive traversal, signal handling, timing, terminal formatting and error handling.

The scope is narrower than a full process manager. It does not provide CPU and memory usage, process control, scheduling changes, filtering by user or historical monitoring. Those capabilities should not be attributed to the implementation unless added and verified.

The educational value is in connecting operating-system concepts to observable system data. Students can trace the lifecycle of one snapshot, understand how parent identifiers create a hierarchy, and evaluate how changing process state affects a live display.

Appendix — Conclusion and Final Review

The Linux Process Tree Visualizer is a C command-line application that collects selected process metadata from Linux procfs and displays it as a coloured parent-child tree. Its main path is to enumerate numeric /proc entries, parse stat records, apply the selected filter, sort by PID, count states, identify roots, recursively render descendants and repeat after a delay in live mode.

The code includes defensive practices such as bounded buffers, validation of parsed fields, skipping records that cannot be read, a printed marker, a recursion-depth guard and a minimal signal handler. Its main limitations are fixed capacity, best-effort snapshots, repeated linear scans, fixed refresh interval and terminal-dependent formatting.

Before final submission, run the documented commands on Linux, record the actual environment, capture authentic screenshots and update the test results. This report intentionally avoids claiming unperformed tests or fabricated output.

Appendix — Source File Review Checklist

Review the submitted source line by line. This appendix does not substitute for the source file; it identifies functions and behaviours to verify in the exact version submitted for assessment.

Confirm the POSIX feature-test macro precedes headers, the required headers are included, constants are defined, and the Process structure contains PID, PPID, name, state and printed marker. Confirm helper functions are present and the compile command succeeds.

If the submitted file differs from the code described here, update the report so it reflects the actual source rather than this checklist.

Source area	**Review point**	**Checked**
Constants and Process struct	Limits, fields and state marker	[ ]
handle_signal()	Only updates running flag	[ ]
read_process()	Parses name, state, PPID and PID	[ ]
scan_processes()	Numeric entries, filter, close directory	[ ]
compare_processes()	Ascending PID comparison	[ ]
find_process()	Returns index or -1	[ ]
print_node()	Depth guard, printed marker, recursion	[ ]
display_tree()	Counts, roots, fallback pass, legend	[ ]
main()	Flags, signals, loop, sleep and exit	[ ]

Appendix — Submission Evidence Checklist

The final academic submission should separate documentation from evidence. A description explains intended behaviour, while test evidence shows what happened when the program was compiled and executed. Both are useful but are not interchangeable.

Insert genuine screenshots from the Linux terminal. Each screenshot should have a figure number and caption. Keep the source code in an appendix or repository and ensure the version shown in the report matches the version used for testing.

Before printing, refresh the table of contents and verify page numbering. Check student names, roll numbers, guide name, campus address and academic year against the supplied sample. Leave signature areas blank until the relevant people sign them.

Evidence item	**Ready**
Title page and front matter checked	[ ]
Student names and roll numbers preserved	[ ]
Guide and institutional details checked	[ ]
Diagrams have captions and readable labels	[ ]
Test outcomes reflect actual execution	[ ]
Terminal screenshots are genuine	[ ]
Table of contents and page numbering refreshed	[ ]
Word and PDF content matches	[ ]
Final page count checked after conversion	[ ]